BÁO CÁO THỰC HÀNH

PHÂN ĐOẠN NGỮ NGHĨA ẢNH GIAO THÔNG BẰNG U-NET TRÊN CAMVID

Học phần: CO5085 – Deep Learning and Its Applications in Computer Vision

Sinh viên: Bùi Tiến Đức MSSV: 2680018

Ngày thực hành: 19/10/2026

Nguồn dataset : https://github.com/stonewst/CamVid-for-Segmentation

Trạng thái: BẢN BÁO CÁO CÓ VỊ TRÍ CHÈN MINH CHỨNG – cập nhật số liệu sau khi chạy lại 10 epochs.

HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

Chạy lần lượt các lệnh bên dưới trong CMD tại thư mục A2. Dùng Win + Shift + S để chụp vùng CMD hoặc hình kết quả; dán vào vị trí "CHÈN ẢNH". Không thay kết quả lần chạy mới bằng số liệu cũ. Giữ nguyên dataset.py đã kiểm tra thành công.

1. Mục tiêu và phạm vi

Xây dựng pipeline phân đoạn ngữ nghĩa ảnh theo nhãn từng pixel trên bộ dữ liệu CamVid: chuẩn bị dữ liệu, ánh xạ mask RGB sang class ID, huấn luyện U-Net và đánh giá bằng mean Intersection over Union (mIoU), Mean Dice. Nội dung này là thực hành Segmentation; A2.1 (chọn đề tài/bài báo) được trình bày riêng ở mục 9, không đồng nhất với A1 CNN–Transformer.

2. Môi trường thực nghiệm

Thành phần	Cấu hình đã xác nhận
Hệ điều hành	Windows 11
Python	3.11 (.venv)
PyTorch	2.11.0+cu128
GPU	NVIDIA GeForce RTX 3050 Ti Laptop GPU (4 GB)
CUDA PyTorch	12.8; torch.cuda.is_available() = True
CUDA Toolkit hệ thống	13.0 (nvcc)

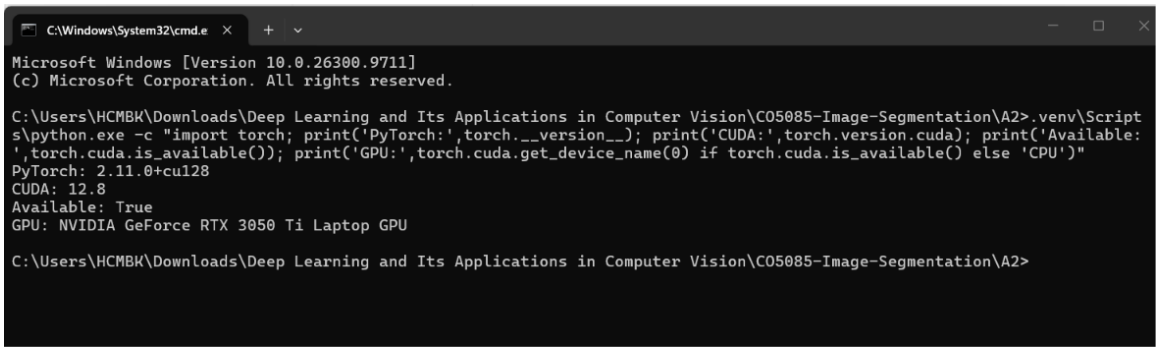
Tại CMD trong thư mục A2, chạy:

```
.venv\Scripts\python.exe -c "import torch; print('PyTorch:',torch.__version__);
print('CUDA:',torch.version.cuda); print('Available:',torch.cuda.is_available());
print('GPU:',torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU')"
```

Nếu lệnh trên gặp lỗi dấu nháy trong CMD, dùng lệnh đơn giản:

```
.venv\Scripts\python.exe -c "import torch; print(torch.__version__, torch.version.cuda,
torch.cuda.is_available())"
```

Hình 1. Xác nhận môi trường PyTorch và CUDA.



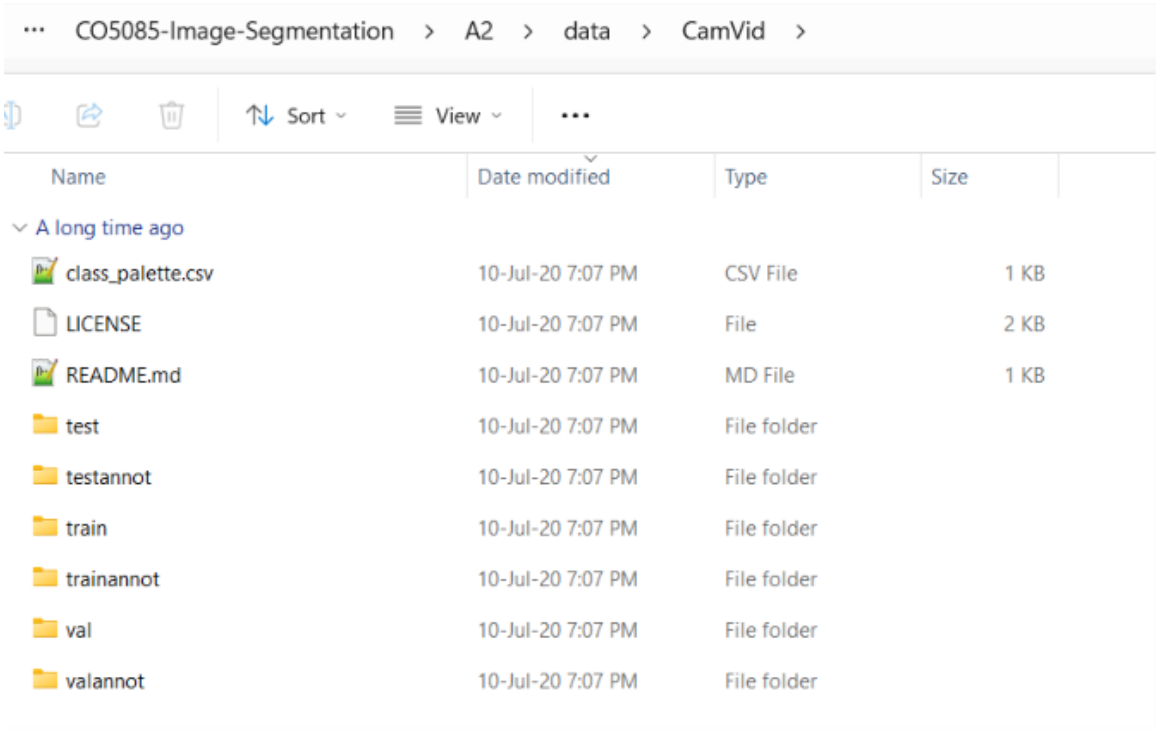
3. Bộ dữ liệu CamVid

Bộ dữ liệu gồm ảnh RGB cảnh đường phố và mask phân đoạn màu. Thực nghiệm sử dụng 11 lớp: Sky, Building, Pole, Road, Sidewalk, Tree, SignSymbol, Fence, Car, Pedestrian, Bicyclist. Vùng unlabeled được mã hóa thành 255 và loại khỏi loss/metrics.

Tập	Số ảnh
Train	367
Validation	101
Test	233
Tổng	701

```
dir data\CamVid
```

Hình 2. Cấu trúc dữ liệu gồm train/trainannot, val/valannot, test/testannot.



4. Tiền xử lý và kiểm tra dữ liệu

Ảnh được resize 256 × 256 bằng bilinear và chuẩn hóa về [0,1]. Mask được resize bằng nearest-neighbor để không tạo nhãn mới; ánh xạ RGB sang class ID 0–10. Mask có kích thước [B,H,W], kiểu long; vùng không gán nhãn = 255.

```
.venv\Scripts\python.exe dataset.py
```

Hình 3. Kết quả kiểm tra ba tập dữ liệu: All splits validated successfully!

```
C:\Users\HCMBK\Downloads\Deep Learning and Its Applications in Computer Vision\CO5085-Image-Segmentation\A2>.venv\Scripts\python.exe dataset.py
============================================================
CAMVID DATASET VALIDATION
============================================================

Split: train
Number of images: 367
Number of classes: 11
Class names: ['Sky', 'Building', 'Pole', 'Road', 'Sidewalk', 'Tree', 'SignSymbol', 'Fence', 'Car', 'Pedestrian', 'Bicyclist']
Images shape: torch.Size([2, 3, 256, 256])
Masks shape: torch.Size([2, 256, 256])
Mask IDs: [0, 1, 2, 3, 4, 5, 6, 8, 9, 255]
Dataset validation: PASSED

Split: val
Number of images: 101
Number of classes: 11
Class names: ['Sky', 'Building', 'Pole', 'Road', 'Sidewalk', 'Tree', 'SignSymbol', 'Fence', 'Car', 'Pedestrian', 'Bicyclist']
Images shape: torch.Size([2, 3, 256, 256])
Masks shape: torch.Size([2, 256, 256])
Mask IDs: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 255]
Dataset validation: PASSED

Split: test
Number of images: 233
Number of classes: 11
Class names: ['Sky', 'Building', 'Pole', 'Road', 'Sidewalk', 'Tree', 'SignSymbol', 'Fence', 'Car', 'Pedestrian', 'Bicyclist']
Images shape: torch.Size([2, 3, 256, 256])
Masks shape: torch.Size([2, 256, 256])
Mask IDs: [0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 255]
Dataset validation: PASSED

All splits validated successfully!

C:\Users\HCMBK\Downloads\Deep Learning and Its Applications in Computer Vision\CO5085-Image-Segmentation\A2>
```

Chụp thêm một cặp ảnh gốc và mask bằng đoạn Python bên dưới. Tạo file show_camvid.py trong A2:

```
from pathlib import Path
from PIL import Image
import matplotlib.pyplot as plt
```

```
root = Path("data/CamVid")
image_path = sorted((root / "val").glob("*.png"))[0]
mask_path = root / "valannot" / (image_path.stem + "_L.png")
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].imshow(Image.open(image_path).convert("RGB"))
ax[0].set_title("Input image")
ax[1].imshow(Image.open(mask_path).convert("RGB"))
ax[1].set_title("Ground truth RGB mask")
for a in ax: a.axis("off")
plt.tight_layout()
Path("results").mkdir(exist_ok=True)
plt.savefig("results/camvid_sample.png", dpi=180)
plt.show()
.venv\Scripts\python.exe show_camvid.py
```

Hình 4. Ảnh đầu vào và mask nhãn thật CamVid (results/camvid_sample.png).

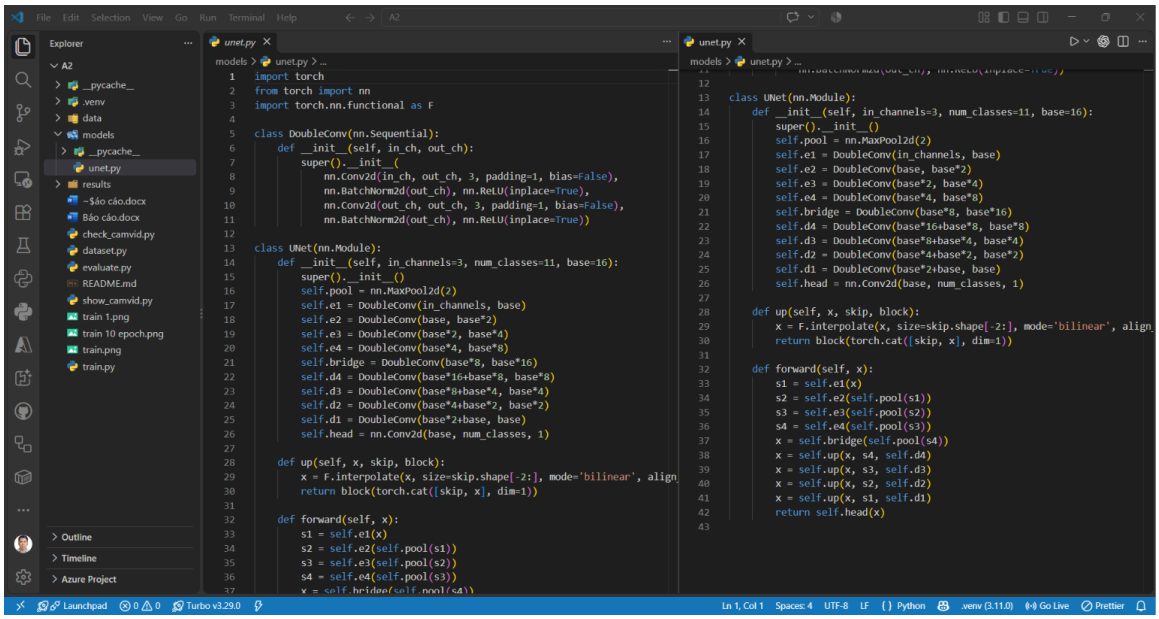


5. Kiến trúc và phương pháp

U-Net gồm encoder trích xuất đặc trưng, bottleneck, decoder khôi phục độ phân giải và skip connections truyền thông tin không gian. Tầng đầu ra dự đoán logits cho 11 lớp tại từng pixel. Hàm loss: CrossEntropyLoss(ignore_index=255). Đối chiếu chi tiết số kênh và optimizer với mã train.py/models/unet.py đang sử dụng trước khi ghi vào bản nộp.

Có thể chụp hình cấu trúc mã nguồn trong VS Code, gồm class UNet và forward(), để làm minh chứng cài đặt. Không dùng sơ đồ U-Net tiêu chuẩn để khẳng định đó là cấu trúc chính xác nếu chưa đối chiếu mã.

Hình 5. Đoạn mã kiến trúc U-Net trong models/unet.py (ảnh chụp VS Code).



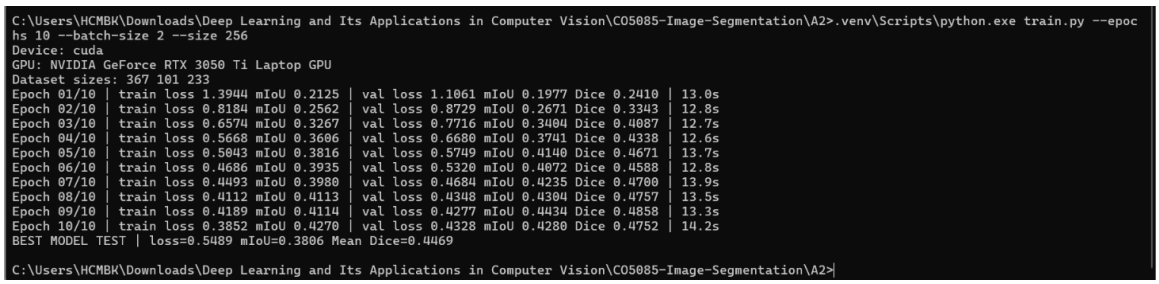
6. Huấn luyện lại 10 epochs

Lưu ý: chạy lại cùng lệnh có thể ghi đè best_unet.pt, history.csv, prediction.png và metrics cũ. Nếu cần giữ kết quả trước, sao lưu thư mục results trước khi chạy.

```
.venv\Scripts\python.exe train.py --epochs 10 --batch-size 2 --size 256
```

Ghi nhận các thông số thực tế xuất ra: device, GPU, số ảnh, train loss, validation loss, validation mIoU, Dice và thời gian/epoch. Chụp toàn bộ log từ epoch 01/10 đến 10/10 và dòng BEST MODEL TEST.

Hình 6. Nhật ký huấn luyện U-Net trong 10 epochs và kết quả test.



7. Đánh giá định lượng và biểu đồ

IoU = TP/(TP+FP+FN); Dice = 2TP/(2TP+FP+FN). mIoU và Mean Dice được tính trung bình theo lớp theo quy tắc của evaluate.py. Cần xác nhận code đã loại pixel 255 và quy tắc xử lý lớp vắng mặt.

Bảng dưới đây là số liệu LẦN CHẠY TRƯỚC để đối chiếu. Thay bằng kết quả chạy lại trước khi nộp.

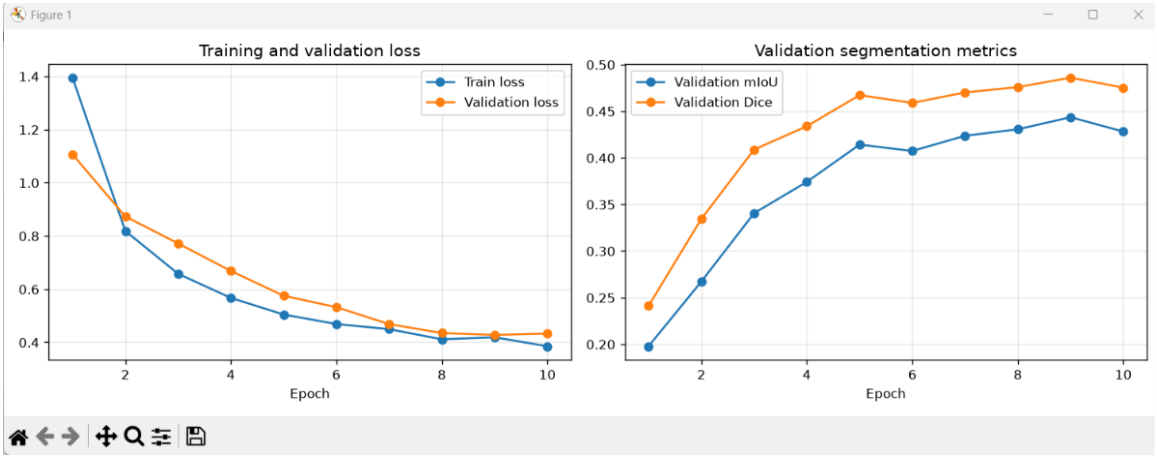
Chỉ số	Lần chạy trước	Lần chạy mới (điền)
Best validation mIoU	0,4337 (epoch 8)	……

Test loss	0,5625	.....
Test mIoU	0,3797	.....
Test Mean Dice	0,4452	.....

Tạo file plot_history.py để vẽ biểu đồ. Script tự in tên các cột CSV; nếu tên cột không trùng các phương án bên dưới, cần sửa theo header thực tế.

```
import csv
from pathlib import Path
import matplotlib.pyplot as plt
path = Path("results/history.csv")
with path.open(encoding="utf-8-sig", newline="") as f:
    rows = list(csv.DictReader(f))
if not rows:
    raise ValueError("history.csv is empty")
print("CSV columns:", list(rows[0]))
def col(*names):
    for name in names:
        if name in rows[0]:
            return [float(r[name]) for r in rows]
    return None
x = col("epoch", "Epoch") or list(range(1, len(rows)+1))
series = [
    ("train_loss", "train_loss", "Train loss"),
    ("val_loss", "val_loss", "Validation loss"),
    ("val_miou", "val_miou", "Validation mIoU"),
    ("val_dice", "val_dice", "Validation Dice"),
]
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
found = False
for _, key, label in series[:2]:
    y = col(key, key.replace("_", " "))
    if y is not None:
        axes[0].plot(x, y, marker="o", label=label)
        found = True
for _, key, label in series[2:]:
    y = col(key, key.replace("_", " "))
    if y is not None:
        axes[1].plot(x, y, marker="o", label=label)
        found = True
if not found:
    raise ValueError("Update column names to match printed CSV columns")
for ax in axes:
    ax.set_xlabel("Epoch")
    ax.grid(True, alpha=0.3)
    ax.legend()
axes[0].set_title("Training and validation loss")
axes[1].set_title("Validation segmentation metrics")
plt.tight_layout()
plt.savefig("results/training_curves.png", dpi=180)
plt.show()
.venv\Scripts\python.exe plot_history.py
```

Hình 7. Biểu đồ loss và chỉ số validation qua 10 epochs (results/training_curves.png).

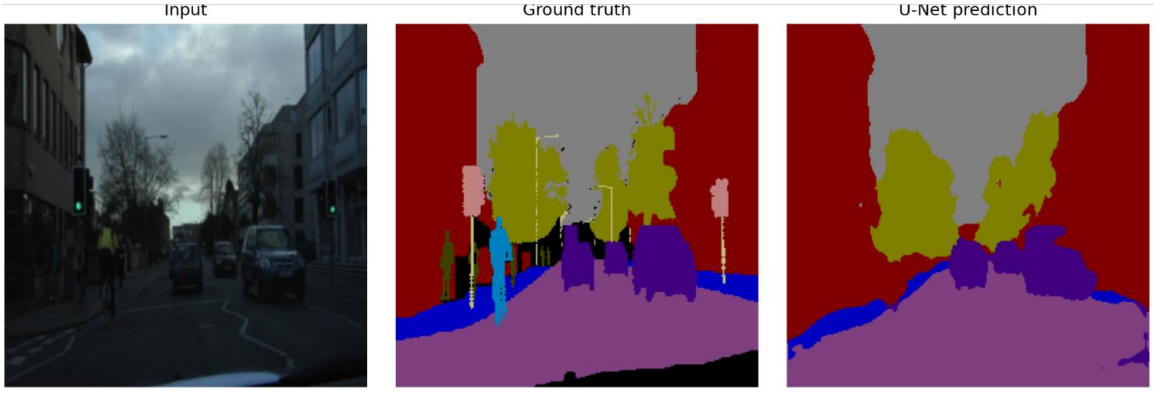


8. Trực quan hóa dự đoán

Chạy evaluate.py để sinh ảnh dự đoán từ checkpoint. Mở ảnh prediction.png và chụp so sánh ảnh đầu vào, ground truth, predicted mask. Chỉ nhận xét các lớp/vùng dự đoán đúng sai sau khi quan sát ảnh thật.

```
.venv\Scripts\python.exe evaluate.py
start results\prediction.png
```

Hình 8. So sánh ảnh gốc, ground truth và predicted mask (results/prediction.png).



9. A2.1 – Đề xuất đề tài và bài báo

Tên đề tài đề xuất: “So sánh U-Net và UNet++ cho phân đoạn ngữ nghĩa cảnh giao thông đô thị trên CamVid”. Bài báo tham chiếu: Zhou, Z. và cộng sự (2020), “UNet++: Redesigning Skip Connections to Exploit Multiscale Features in Image Segmentation”, IEEE Transactions on Medical Imaging, DOI: 10.1109/TMI.2019.2959609. Đây là định hướng ứng dụng kiến trúc UNet++ trên CamVid, không phải tái lập đúng thí nghiệm ảnh y sinh của bài báo.

Câu hỏi nghiên cứu: (1) U-Net đạt mIoU/Dice bao nhiêu? (2) UNet++ có cải thiện dưới cùng quy trình huấn luyện không? (3) Đánh đổi giữa chất lượng phân đoạn và thời gian/bộ nhớ GPU ra sao? Phần UNet++ chưa có kết quả, không được trình bày như đã hoàn thành.

10. Kết luận và giới hạn

Pipeline U-Net + CamVid đã vận hành được trên GPU và hoàn thành kiểm tra dữ liệu. Kết quả chạy lại 10 epochs sẽ được điền vào bảng và minh chứng hình 6–8. Cần xem xét ảnh dự đoán, mất cân bằng lớp và các biến động validation trước khi kết luận về chất lượng mô hình.

11. Checklist trước khi nộp GitHub

☐ Chạy lại 10 epochs và ghi số liệu mới; kiểm tra checkpoint đã tạo.

☐ Chạy evaluate.py; chèn prediction.png thật.

☐ Chạy show_camvid.py và plot_history.py; chèn hình 4, 7.

☐ Chèn ảnh CMD môi trường, dataset validation, training log.

☐ Kiểm tra train.py, evaluate.py và models/unet.py khớp mô tả.

☐ Upload code, README, biểu đồ, báo cáo; không upload .venv, dataset hoặc checkpoint lớn.

☐ Ghi rõ việc sử dụng AI hỗ trợ nếu học phần yêu cầu.

Tài liệu tham khảo

1. Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI.

2. Brostow, G. J., Fauqueur, J., & Cipolla, R. (2009). Semantic Object Classes in Video: A High-Definition Ground Truth Database. Pattern Recognition Letters.

3. Zhou, Z., et al. (2020). UNet++: Redesigning Skip Connections to Exploit Multiscale Features in Image Segmentation. IEEE Transactions on Medical Imaging. DOI: 10.1109/TMI.2019.2959609.